

Gas Storage Contract Pricing Model Using Python

Introduction

Natural gas companies often purchase gas when market prices are low and store it for future sale when prices increase. This process helps companies maximize profit through strategic buying and selling decisions.

The objective of this project is to build a prototype pricing model that calculates the value of a gas storage contract using Python. The model considers various business constraints such as storage limits, injection and withdrawal rates, and storage costs.

Objective

The main objective of this project is to create a pricing function that:

- Calculates the value of a gas storage contract
 - Handles multiple injection and withdrawal dates
 - Considers gas purchase and selling prices
 - Includes storage costs and capacity constraints
 - Provides the final profit or contract value
-

Problem Statement

A client wants to store natural gas by purchasing it at lower prices and selling it later at higher prices. The company needs a pricing model that can estimate whether the contract will generate profit.

The pricing model should consider:

- Injection dates
 - Withdrawal dates
 - Gas prices
 - Injection and withdrawal rates
 - Maximum storage capacity
 - Storage costs
-

Technologies Used

- Python
 - Pandas
 - NumPy
-

Business Logic

The model follows these steps:

1. Buy gas during injection dates
 2. Store gas in storage facility
 3. Pay storage costs
 4. Withdraw gas during withdrawal dates
 5. Sell gas at higher prices
 6. Calculate final profit
-

Assumptions

- No transportation delay
 - Interest rates are zero
 - Market holidays are ignored
 - Gas can be injected and withdrawn instantly within rate limits
-

Formula Used

Purchase Cost

Purchase Cost = Injected Volume × Injection Price

Revenue

Revenue = Withdrawn Volume × Withdrawal Price

Final Contract Value

Final Profit = Total Revenue – Purchase Cost – Storage Cost

Python Implementation

```
import pandas as pd
import numpy as np

def price_gas_contract(
    injection_dates,
    withdrawal_dates,
    injection_prices,
    withdrawal_prices,
    injection_rate,
    withdrawal_rate,
    max_storage_volume,
    storage_cost_per_unit):

    total_profit = 0
    stored_volume = 0

    # Injection phase
    for i in range(len(injection_dates)):

        injected_volume = min(
            injection_rate,
            max_storage_volume - stored_volume
        )

        stored_volume += injected_volume

        purchase_cost = injected_volume * injection_prices[i]

        total_profit -= purchase_cost

        total_profit -= injected_volume * storage_cost_per_unit

    # Withdrawal phase
    for i in range(len(withdrawal_dates)):

        withdrawn_volume = min(
            withdrawal_rate,
            stored_volume
        )

        stored_volume -= withdrawn_volume

        sale_revenue = withdrawn_volume * withdrawal_prices[i]
```

```
        total_profit += sale_revenue

    return total_profit
```

Sample Input

```
injection_dates = ["2024-01-01", "2024-02-01"]

withdrawal_dates = ["2024-06-01", "2024-07-01"]

injection_prices = [20, 22]

withdrawal_prices = [28, 30]

injection_rate = 1000

withdrawal_rate = 1000

max_storage_volume = 2000

storage_cost_per_unit = 1
```

Output

```
Contract Value: 14000
```

Result Analysis

The model generated a positive contract value because gas was purchased at lower prices and sold later at higher prices.

The storage cost reduced the total profit, but the selling prices were high enough to generate overall gain.

This demonstrates how companies can use pricing models to make strategic storage and trading decisions.

Key Learnings

Through this project, the following concepts were learned:

- Python functions
 - Financial modeling
 - Business logic implementation
 - Cash flow calculations
 - Storage optimization
 - Data analysis thinking
-

Conclusion

This project successfully created a prototype gas storage pricing model using Python. The model can estimate the value of a gas storage contract while considering important business constraints such as storage capacity, rates, and costs.

The project demonstrates how Python can be used in financial and energy market analysis to support business decision-making.